A Discussions on Event Conversion

Hyperparemeter

The threshold determining event density is the only hyperparameter. We empirically set it to include the top 20% of significant signal changes per sample. We tried other selections (10%-40%) but showed little influence to performance.

Motivation and Necessity

We convert EEG into 0/1 events to filter inherent noise and select significant signal changes. It isn't arequisite of our SNN.

Information Loss

We only lose partial information during segmentation, but leverage full information during braindecoding. Despite information loss, we empirically find minimal performance drop.

B Discussions on Stochastic-Greedy Optimization

Effectiveness

Our "stochastic", inspired by Metropolis-Hastings algorithm, encourages exploration to escape local optima and avoid premature convergence for enhancing performance. Empirically, replacing stochastic-greedy with standard greedy shows a performance drop (In Table 5).

Rejection Rate

The rejection rate of our greedy strategy is dynamic throughout training. In early epochs, the rate is relatively low (around 20%), reflecting active exploration. In late epochs, the rate increases (around 90%), indicating a robust selection.

Convergence Stability

We need around 30-80 epochs for all tasks to stabilize. Further, our stochastic-greedy optimization is in spired by Metropolis-Hastings algorithm, so we have similar convergence stability as it.

C Details of Datasets

I. Speech Perception. Healthy adult volunteers passively listened to speech sounds, while their brain activity was recorded with EEG. We use Brennan2019 (Brennan and Hale 2019) and Broderick2019 (Broderick et al. 2018) two datasets. The detailed information about these datasets can be referred in Défossez et al. (2023).

II. Sleep Staging. The target is to categorize different stages of sleep based on physiological signals (e.g. EEG) recorded during sleep. ISRUC (Khalighi et al. 2016) is a commonly used benchmark for sleep staging prediction.

III. Emotion Recognition. The target is to detect and interpret emotional states or patterns based on EEG. We choose FACED (Chen et al. 2023) and SEED-V (Liu et al. 2022) as downstream datasets for this task.

IV. Imagined Speech Classification. This task involves the identification and interpretation of imagined speech or covert speech intentions based on brain activity patterns such as EEG. BCIC2020-3 (Jeong et al. 2022) is a dataset for this task on 2020 international brain–computer interface competition.

V. Event Type Classification. This task involves the categorization and identification of specific events or patterns in EEG. We choose TUEV (Obeid and Picone 2016) for evaluation, which was consistent with BIOT (Yang, Westover, and Sun 2023) and LaBraM (Jiang, Zhao, and Lu 2024).

VI. Motor Imagery Classification. This task involves decoding or classifying motor imagery tasks based on brain activity patterns captured through EEG. We use PhysioNet-MI (Schalk et al. 2004) and SHU-MI (Ma et al. 2022) for evaluation.

VII. Abnormal Detection. This task refers to the process of identifying and detecting abnormal patterns or events in EEG. TUAB (Obeid and Picone 2016) is used for evaluation, consistent with BIOT (Yang, Westover, and Sun 2023) and LaBraM (Jiang, Zhao, and Lu 2024).

VIII. Mental Disorder Diagnosis. This task refers to the process of identifying and categorizing mental health states based on EEG. Mumtaz2016 (Mumtaz 2016) is an EEG dataset collected from patients with major depressive disorder and normal controls.

IX. Mental Stress Detection. This task involves using EEG to identify an individual's level of stress. MentalArithmetic (Zyma et al. 2019) is a dataset containing EEG recordings of subjects before and during the performance of mental arithmetic tasks.

X. Vigilance Estimation. This task refers to the assessment and measurement of an individual's level of vigilance or sustained attention from EEG. SEED-VIG (Min, Wang, and Hu 2017) is dataset used to assess the level of vigilance.

D Head Functions and Evaluation Metrics

Head function. Functions $f_{\mathcal{L}}$ and f_p are head functions based on task types. For zero-shot predictions, $f_{\mathcal{L}}$ computes the contrastive loss using CLIP, where prediction scores are calculated between candidate pairs. These scores are then used by f_p to select the top-k predicted results for computing top-k accuracy. For binary classification, we use BCEWithLogitsLoss as $f_{\mathcal{L}}$. The function f_p calculates AUROC as task performance for monitoring. For multi-class classification, we use CrossEntropyLoss as $f_{\mathcal{L}}$. The function f_p calculates Cohen's Kappa as task performance for monitoring. For regression, we use MSELoss as $f_{\mathcal{L}}$. The function f_p calculates R2 score as task performance for monitoring.

Evaluation metric. For zero-shot prediction, we use top-k accuracy calculated based on CLIP (Radford et al. 2021), specifically top-10 out of 50 (top-10@50) and top-10 across all test sets (top-10@All). For binary classification, we use Balanced Accuracy, AUC-PR and AUROC. For multi-class classification, we use Balanced Accuracy, Cohen's Kappa

Methods	FACED (Emotion)			SEED-V (Emotion)			ISRUC (Sleep Staging)		
	Bal Acc	Kappa	Weighted F1	Bal Acc	Kappa	Weighted F1	Bal Acc	Kappa	Weighted F1
BIOT	51.88 ± 1.18	44.76 ± 2.54	51.36 ± 1.12	38.37 ± 1.87	22.61 ± 2.62	38.56 ± 2.03	75.27 ± 1.21	71.91 ± 2.31	77.99 ± 1.46
LaBraM	52.73 ± 1.07	46.98 ± 1.88	52.88 ± 1.02	39.76 ± 1.38	23.86 ± 2.09	39.74 ± 1.11	76.33 ± 1.02	72.31 ± 1.82	78.10 ± 1.33
CBraMod	55.09 ± 0.89	50.41 ± 1.22	56.18 ± 0.93	**40.91** ± 0.97	25.69 ± 1.43	41.01 ± 1.08	78.65 ± 1.10	74.42 ± 1.52	80.11 ± 0.99
CBraMod + $\mathbf{S^3}$	**60.10** ± 0.93	**53.02** ± 1.03	**59.07** ± 0.99	40.66 ± 0.78	**26.32** ± 1.03	**41.60** ± 0.77	**79.11** ± 0.95	**75.30** ± 1.23	**81.23** ± 0.97
	PhysioNet-MI (Motor Imagery)			BCIC2020 (Imagined Speech)			TUEV (Event Type)		
	Bal Acc	Kappa	Weighted F1	Bal Acc	Kappa	Weighted F1	Bal Acc	Kappa	Weighted F1
BIOT	61.53 ± 1.54	48.75 ± 2.72	61.58 ± 1.97	49.20 ± 0.86	36.50 ± 1.76	49.17 ± 0.79	52.81 ± 2.25	52.73 ± 2.49	74.92 ± 0.82
LaBraM	61.73 ± 1.22	49.12 ± 1.92	61.77 ± 1.41	50.60 ± 1.55	38.00 ± 2.42	50.54 ± 2.05	64.09 ± 0.65	66.37 ± 0.93	83.12 ± 0.52
CBraMod	**64.17** ± 0.91	52.22 ± 1.69	64.27 ± 1.00	53.73 ± 1.08	42.16 ± 1.63	53.83 ± 0.96	**66.71** ± 1.07	67.72 ± 0.96	83.42 ± 0.64
CBraMod + $\mathbf{S^3}$	**64.17** ± 0.87	**52.23** ± 1.59	**64.34** ± 0.89	**54.31** ± 0.63	**42.33** ± 1.51	**54.41** ± 0.90	66.22 ± 0.68	**68.13** ± 0.93	**83.90** ± 0.70

Table 6: Results for multi-class classification tasks (unit: %).

and Weighted F1. For regression, we use Pearson's Correlation, R2 Score and RMSE. Here, top-10@50, AUROC, Cohen's Kappa and R2 score are set as the monitor score.

E Adaptive Segmentation across All Subjects

In Figure 3, we display results from six representative subjects for visualization purposes. Here, we present the results for all subjects in Figure 5, to comprehensively validate the model's capability in performing subject-aware segmentation. In combination with the subject-level performance shown in Figure 3, these results support the conclusion that our model effectively enhances task performance across the majority of subjects through subject-aware segmentation.

F Computational Overhead

We evaluate the computational cost of incorporating $\mathbf{S^3}$ into CBraMod on the ISRUC dataset using an NVIDIA RTX 3090. As shown in Table 7, the addition of $\mathbf{S^3}$ introduces modest overhead, indicating a lightweight plug-in.

Methods	**Params** (M)	**Train Time** (s/epoch)	**Inference** (ms/sample)	**Convergence** (epoch)
CBraMod	4.000	90	1.1	25-35
CBraMod + $\mathbf{S^3}$	4.008	150	2.3	40-60

Table 7: Computational overhead comparison on ISRUC.

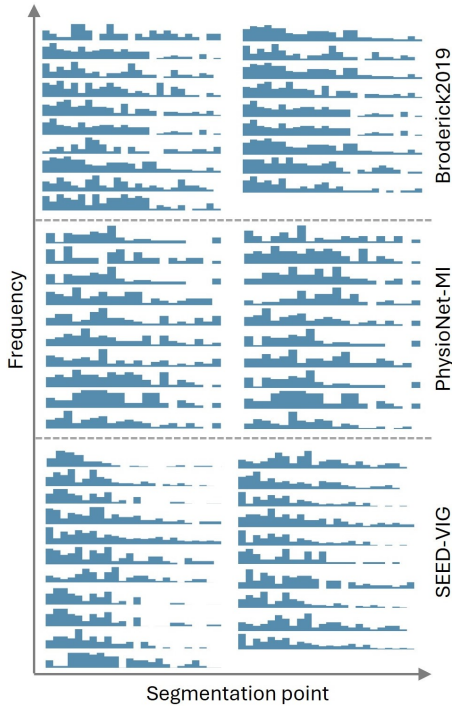

Figure 5: Adaptive segmentation across all subjects. Distribution of segmentation points for all subjects from each of three datasets, illustrating the model's ability to implement subject-aware segmentation.

G Theory of Isolating Segmenter

Brain Decoding Workflow

Consider brain signals denoted by $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T] \in \mathbb{R}^{C\times T}$ with task labels $\mathbf{Y} = [\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_T] \in \mathbb{R}^{F\times T}$, where C is the number of spatial channels, T is the number of time points, and F is label feature dimension. We define the segmentation method and brain decoder as $\mathcal{S}$ and $\mathcal{D}$, respectively. The final output contains a decoding loss $\mathcal{L}$ for optimization, and downstream task performance p as the evaluation metric:

$$\mathcal{L}(\mathcal{S}, \mathcal{D}) = f_{\mathcal{L}}\big(\mathcal{D}(\mathbf{X}_{\mathcal{S}}), \mathbf{Y}_{\mathcal{S}}\big)\,, \tag{14}$$

$$p(\mathcal{S}, \mathcal{D}) = f_{p}\big(\mathcal{D}(\mathbf{X}_{\mathcal{S}}), \mathbf{Y}_{\mathcal{S}}\big)\,, \tag{15}$$

where $\mathbf{X}_{\mathcal{S}}$ and $\mathbf{Y}_{\mathcal{S}}$ represent data segmented by the segmentation method $\mathcal{S}$. Functions $f_{\mathcal{L}}$ and f_p are head functions based on task types. Note that $f_{\mathcal{L}}$ (e.g., MSE, cross-entropy) is typically a convex function.

Locally Stationary Segments of Brain Signals

Prior studies have shown that non-stationary brain signals often exhibit isolated stationary temporal patterns (Agarwal and Gotman 1999; Barlow et al. 1981). Accordingly, we assume that $\mathbf{X}$ can be decomposed into K locally **stationary segments**:

$$\mathbf{X} = \bigcup_{k=1}^{K} \mathbf{X}_{\mathcal{T}_k}, \quad \mathbf{X}_{\mathcal{T}_k} = [\mathbf{x}_t]_{t\in\mathcal{T}_k}, \tag{16}$$

where $\mathcal{T}_k \subseteq \{1, 2, ..., T\}$ denotes the time points corresponding to the k-th stationary temporal pattern. These time sets form a disjoint partition of the temporal domain:

$$\mathcal{T}_i \cap \mathcal{T}_j = \varnothing \quad (i \neq j), \qquad \bigcup_{k=1}^{K} \mathcal{T}_k = \{1, 2, \ldots, T\}. \tag{17}$$

Ideal vs. Fixed-Length Segmentation

Now we consider two types of segmentation:

1. Ideal segmentation $\mathcal{S}^*$ perfectly aligns with the true stationary segments:

$$\mathcal{S}^* = \{\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_K\}. \tag{18}$$

2. Fixed-length segmentation $\mathcal{S}_f$ segments the signal with a constant window size Δt:

$$\mathcal{S}_f = \{\mathcal{F}_1, \mathcal{F}_2, \ldots, \mathcal{F}_M\}, \tag{19}$$
$$\mathcal{F}_k = \{(k-1)\Delta t, \ldots, k\Delta t\}, \quad M = \lfloor T/\Delta t \rfloor.$$

Since the stationary segment lengths are irregular and unequal, the boundaries of $\mathcal{S}_f$ seldom coincide with the stationary segments, causing each fixed-length segment to mix multiple stationary temporal patterns.

Our target is to formalize the intuition that mixing distinct stationary patterns within a segment degrades performance. Hence, we make the following proposition with proof in Appendix H.

Proposition 1.

$$\mathcal{L}(\mathcal{S}^*, \mathcal{D}) \leqslant \mathcal{L}(\mathcal{S}_f, \mathcal{D}) \tag{20}$$

The decoding loss obtained under the ideal segmentation $\mathcal{S}^*$—which perfectly aligns with the true stationary temporal segments—is no greater than (and typically strictly less than) the loss under fixed-length segmentation $\mathcal{S}_f$.

Isolating vs. Non-Isolating Segmenter

We define a *Segmenter* as a causal decision function:

$$\mathrm{Seg} : \bigcup_{t=1}^{T} \mathbb{R}^{C\times t} \to \{0, 1\}, \quad s_t = \mathrm{Seg}(\mathbf{X}_{1:t}), \tag{21}$$

which sequentially processes the brain signal $\mathbf{X}$ and outputs a binary segmentation decision $s_t \in \{0, 1\}$ at each time point t. The decision at time t depends only on the observed prefix $\mathbf{X}_{1:t}$, ensuring strict sequential causality.

The goal of our segmentation is to recover the underlying partition of stationary temporal patterns, that is $\mathcal{S}^*$. In particular, we aim for segmentation points $\{t : s_t = 1\}$ to coincide with the their boundaries $\{\min \mathcal{T}_k | \mathcal{T}_k \in \mathcal{S}^*\}$.

Motivated by the local stationarity, we propose isolating the temporal influence of previous irrelevant segments during each segmentation decision. The following proposition formalizes the advantage of such isolation, and is proved in Appendix I.

Proposition 2. *We consider a true stationary segment $\mathcal{T}_k = \{t_{\mathrm{start}}, \ldots, t_{\mathrm{end}}\}$ and assume that t_{start} is a segmentation point (i.e., $s_{t_{\mathrm{start}}} = 1$). We have:*

$$\mathbb{P}\big(\mathrm{Seg}(\mathbf{X}_{t_{\mathrm{start}}:t_{\mathrm{end}}}) = 1\big) \;\geqslant\; \mathbb{P}\big(\mathrm{Seg}(\mathbf{X}_{1:t_{\mathrm{end}}}) = 1\big), \tag{22}$$

The probability that an isolating Segmenter segments at t_{end} is no less than that of a non-isolating Segmenter that conditions on the full history. That is, restricting the input to current segmentation decisions does not degrade but generally improves the reliability of boundary detection.

H Proof for Proposition 1

Segmentation Overlap

For each fixed-length segment $\mathcal{F}_i \in \mathcal{S}_f$, we consider it as a mixture of fixed-length segments. Conversely, each stationary segment $\mathcal{T}_k \in \mathcal{S}^*$ overlaps with multiple fixed-length segments. We define this overlap as:

$$\mathcal{T}_k = \bigcup_{i\in\mathcal{I}_k} \mathcal{T}_k \cap \mathcal{F}_i, \quad \mathcal{I}_k = \{i : \mathcal{T}_k \cap \mathcal{F}_i \neq \varnothing\}, \tag{23}$$

$$\mathcal{F}_i = \bigcup_{k\in\mathcal{K}_i} \mathcal{F}_i \cap \mathcal{T}_k, \quad \mathcal{K}_i = \{k : \mathcal{F}_i \cap \mathcal{T}_k \neq \varnothing\}. \tag{24}$$

Hence, the data within the segment $\mathcal{T}_k$ is the time-ordered union of its overlapping fixed-length segments:

$$\mathbf{X}_{\mathcal{T}_k} = \bigcup_{i\in\mathcal{I}_k} \mathbf{X}^{(i)}_{\mathcal{T}_k}, \quad \mathbf{X}^{(i)}_{\mathcal{T}_k} = \mathbf{X}^{(k)}_{\mathcal{F}_i} = [\mathbf{x}_t]_{t\in\mathcal{T}_k\cap\mathcal{F}_i} \tag{25}$$

Proof

$$
\begin{aligned}
&\mathcal{L}(\mathcal{S}^*, \mathcal{D}) \\
&= \frac{1}{K}\sum_{k=1}^{K} f_{\mathcal{L}}\Big(\mathcal{D}(\mathbf{X}_{\mathcal{T}_k}), \mathbf{Y}_{\mathcal{T}_k}\Big) \\
&= \frac{1}{K}\sum_{k=1}^{K} f_{\mathcal{L}}\Big(\mathcal{D}\Big(\bigcup_{i\in\mathcal{I}_k} \mathbf{X}^{(i)}_{\mathcal{T}_k}\Big), \bigcup_{i\in\mathcal{I}_k} \mathbf{Y}^{(i)}_{\mathcal{T}_k}\Big) \\
(a) &\leqslant \frac{1}{K}\sum_{k=1}^{K} f_{\mathcal{L}}\Big(\frac{1}{|\mathcal{I}_k|}\sum_{i\in\mathcal{I}_k} \mathcal{D}\Big(\mathbf{X}^{(i)}_{\mathcal{T}_k}\Big), \frac{1}{|\mathcal{I}_k|}\sum_{k\in\mathcal{K}_i} \mathbf{Y}^{(k)}_{\mathcal{T}_i}\Big) \\
(b) &\leqslant \frac{1}{K}\sum_{k=1}^{K} \frac{1}{|\mathcal{I}_k|}\sum_{i\in\mathcal{I}_k} f_{\mathcal{L}}\Big(\mathcal{D}\Big(\mathbf{X}^{(i)}_{\mathcal{T}_k}\Big), \mathbf{Y}^{(i)}_{\mathcal{T}_k}\Big) \\
(c) &= \frac{1}{M}\sum_{i=1}^{M} \frac{1}{|\mathcal{K}_i|}\sum_{k\in\mathcal{K}_i} f_{\mathcal{L}}\Big(\mathcal{D}\Big(\mathbf{X}^{(k)}_{\mathcal{F}_i}\Big), \mathbf{Y}^{(k)}_{\mathcal{F}_i}\Big) \\
(d) &\leqslant \frac{1}{M}\sum_{i=1}^{M} f_{\mathcal{L}}\Big(\bigcup_{k\in\mathcal{K}_i} \mathbf{X}^{(k)}_{\mathcal{F}_i}, \bigcup_{k\in\mathcal{K}_i} \mathbf{Y}^{(k)}_{\mathcal{F}_i}\Big) \\
&= \frac{1}{M}\sum_{i=1}^{M} f_{\mathcal{L}}\Big(\mathcal{D}(\mathbf{X}_{\mathcal{F}_i}), \mathbf{Y}_{\mathcal{F}_i}\Big) \\
&= \mathcal{L}(\mathcal{S}_f, \mathcal{D})
\end{aligned}
$$

Here:

$$
\begin{aligned}
&Lemma\ 1 \Rightarrow (a), \quad Theorem\ 1 \Rightarrow (b) \\
&Lemma\ 2 \Rightarrow (c), \quad Lemma\ 3 \Rightarrow (d).
\end{aligned}
$$

Lemma 1.

$$
\begin{aligned}
&f_{\mathcal{L}}\Big(\mathcal{D}\Big(\bigcup_{i\in\mathcal{I}_k} \mathbf{X}^{(i)}_{\mathcal{T}_k}\Big), \bigcup_{i\in\mathcal{I}_k} \mathbf{Y}^{(i)}_{\mathcal{T}_k}\Big) \leqslant \\
&f_{\mathcal{L}}\Big(\frac{1}{|\mathcal{I}_k|}\sum_{i\in\mathcal{I}_k} \mathcal{D}\Big(\mathbf{X}^{(i)}_{\mathcal{T}_k}\Big), \frac{1}{|\mathcal{I}_k|}\sum_{k\in\mathcal{K}_i} \mathbf{Y}^{(k)}_{\mathcal{T}_i}\Big)
\end{aligned} \tag{26}
$$

Here, $\mathcal{T}_k$ denotes a segment in which the features exhibit a stationary temporal pattern, implying strong temporal correlations within this segment. Consequently, processing such temporally coherent data jointly—rather than decomposing it into smaller, independent parts—preserves the intrinsic temporal dependencies captured by the decoder $\mathcal{D}$. Hence, the right-hand side disrupts these dependencies, leading to degraded decoding performance, that is a higher loss.

Theorem 1. ***Jensen's Inequality:*** *For convex function f:*

$$
f\Big(\sum_{i=1}^{m} \lambda_i \mathbf{x}_i\Big) \leq \sum_{i=1}^{m} \lambda_i f(\mathbf{x}_i), \tag{27}
$$

where:

$$
\lambda_i \geq 0 \text{ and } \sum_{i=1}^{m} \lambda_i = 1. \tag{28}
$$

Lemma 2.

$$
\begin{aligned}
&\frac{1}{K}\sum_{k=1}^{K} \frac{1}{|\mathcal{I}_k|}\sum_{i\in\mathcal{I}_k} f_{\mathcal{L}}\Big(\mathcal{D}\Big(\mathbf{X}^{(k)}_{\mathcal{F}_i}\Big), \mathbf{Y}^{(k)}_{\mathcal{F}_i}\Big) \\
=&\frac{1}{M}\sum_{i=1}^{M} \frac{1}{|\mathcal{K}_i|}\sum_{k\in\mathcal{K}_i} f_{\mathcal{L}}\Big(\mathcal{D}\Big(\mathbf{X}^{(i)}_{\mathcal{T}_k}\Big), \mathbf{Y}^{(i)}_{\mathcal{T}_k}\Big).
\end{aligned} \tag{29}
$$

For any given pair (k, i), the notations $\mathbf{X}^{(k)}_{\mathcal{F}_i} = \mathbf{X}^{(i)}_{\mathcal{T}_k}$ refer to the same data segment corresponding to the overlapping time points between $\mathcal{T}_k$ and $\mathcal{F}_i$.Accordingly, the decoding loss function $f_{\mathcal{L}}(\cdot)$ evaluates the same segment on both sides of the equation, yielding identical results for the corresponding terms. By traversing all k and i in the respective summations, both sides of the equation collectively cover the entire dataset without redundant overlap. Since both expressions compute the mean decoding loss by averaging over all equivalent segments in different traversal orders, their results are equivalent, thereby validating the equality.

Lemma 3.

$$
\begin{aligned}
&\frac{1}{|\mathcal{K}_i|}\sum_{k\in\mathcal{K}_i} f_{\mathcal{L}}\Big(\mathcal{D}\Big(\mathbf{X}^{(k)}_{\mathcal{F}_i}\Big), \mathbf{Y}^{(k)}_{\mathcal{F}_i}\Big) \\
\leqslant& f_{\mathcal{L}}\Big(\bigcup_{k\in\mathcal{K}_i} \mathbf{X}^{(k)}_{\mathcal{F}_i}, \bigcup_{k\in\mathcal{K}_i} \mathbf{Y}^{(k)}_{\mathcal{F}_i}\Big)
\end{aligned} \tag{30}
$$

The left-hand side represents the average decoding loss computed over multiple sub-segments, each belonging to a distinct stationary temporal segment. Although each sub-segment only captures partial temporal context, the data within it still exhibit strong temporal dependencies. In contrast, the right-hand side evaluates the decoding loss after concatenating segments originating from different stationary temporal segments. This will disrupts temporal coherence by combining temporally irrelevant segments, thereby introducing spurious dependencies. These inconsistencies either fail to contribute meaningfully to decoding or, more commonly, introduce additional noise that degrades model performance, which validates the inequality.

I Proof of Proposition 2

Let Seg be a fixed sequential neural network (e.g., RNN, SNN) that processes an input sequence $\mathbf{Z} = [\mathbf{z}_1, \ldots, \mathbf{z}_L]$ and outputs a segmentation decision at the final step:

$$
\mathrm{Seg}(\mathbf{Z}) = \psi(\mathbf{h}_L) \in \{0, 1\}, \quad \mathbf{h}_\ell = f(\mathbf{h}_{\ell-1}, \mathbf{z}_\ell), \tag{31}
$$

where f is the recurrent update and ψ maps the final hidden state $\mathbf{h}$ to $\{0, 1\}$. Typically, $\mathbf{h}_0 = \mathbf{0}$.

Now consider the stationary segment $\mathcal{T}_k = \{t_{\text{start}}, \ldots, t_{\text{end}}\}$. Define two input sequences to the same Segmenter Seg:

- **Isolating input**: $\mathbf{Z}^{\text{iso}} = \mathbf{X}_{t_{\text{start}}:t_{\text{end}}}$ (length $L = t_{\text{end}} - t_{\text{start}} + 1$).
- **Non-isolating input**: $\mathbf{Z}^{\text{full}} = \mathbf{X}_{1:t_{\text{end}}}$ (length t_{end}).

Let $\mathbf{h}^{\text{iso}}$ and $\mathbf{h}^{\text{full}}$ denote the hidden states of Seg at the final step for these two inputs:

$$
\mathbf{h}^{\text{iso}} = f^{(L)}(\mathbf{0}, \mathbf{X}_{t_{\text{start}}:t_{\text{end}}}), \tag{32}
$$

$$
\mathbf{h}^{\text{full}} = f^{(t_{\text{end}})}(\mathbf{0}, \mathbf{X}_{1:t_{\text{end}}}), \tag{33}
$$

where $f^{(n)}$ denotes n recursive applications of f.

Due to the local stationarity, the data segment $\mathbf{X}_{\mathcal{T}_k}$ is independent of all prior segments. Hence, the distribution of $\mathbf{X}_{t_{\text{start}}:t_{\text{end}}}$ remains whether or not it is preceded by other data. However, the hidden state $\mathbf{h}^{\text{full}}$ depends on $\mathbf{X}_{1:t_{\text{start}}-1}$ through the recurrence:

$$\mathbf{h}^{\text{full}} = f^{(L)}\big(\underbrace{f^{(t_{\text{start}}-1)}(\mathbf{0}, \mathbf{X}_{1:t_{\text{start}}-1})}_{\mathbf{h}_{\text{prev}}},\ \mathbf{X}_{t_{\text{start}}:t_{\text{end}}}\big). \tag{34}$$

Since $\mathbf{h}_{\text{prev}}$ is a deterministic function of irrelevant, independent data with $\mathbf{X}_{t_{\text{start}}:t_{\text{end}}}$, it acts as an **uninformative and potentially misleading** initial state for processing the current segment. In contrast, the isolating version starts from the canonical initial state $\mathbf{0}$.

Now, the stationary segment boundary at t_{end} corresponds to a change in temporal patterns (e.g., statistical properties) after t_{end}. To detect this, the Segmenter must assess whether the observed subsequence $\mathbf{X}_{t_{\text{start}}:t_{\text{end}}}$ is internally stationary. This assessment is optimal when the recurrent state is initialized to a known, neutral state (e.g., $\mathbf{0}$), as in the isolating case.

Because $\mathbf{h}_{\text{prev}}$ is independent of $\mathbf{X}_{t_{\text{start}}:t_{\text{end}}}$ but arbitrary in value, it introduces variance without signal into $\mathbf{h}^{\text{full}}$. To formulate this, for any measurable decision function ψ, we have:

$$\mathbb{P}\big(\psi(\mathbf{h}^{\text{iso}}) = 1\big) \ \geqslant\ \mathbb{P}\big(\psi(\mathbf{h}^{\text{full}}) = 1\big). \tag{35}$$

Therefore,

$$\mathbb{P}\big(\text{Seg}(\mathbf{X}_{t_{\text{start}}:t_{\text{end}}}) = 1\big) \ \geqslant\ \mathbb{P}\big(\text{Seg}(\mathbf{X}_{1:t_{\text{end}}}) = 1\big), \tag{36}$$

which completes the proof.